Undecidability of One-Dimensional Array Logic

Fredie George

May 2025

1 Introduction

This is a collection of some reductions I was able to come up with for decidability of satisfiablity for array logic. I first present the reduction to the tiling problem, then give a reduction using two PDAs, then give the shortest reduction using a deterministic two-stack machine (This was found after Kamal Lodya suggested me to check out the proof given in [3]).

Throughout, an array is a function on a nonempty domain $\mathbb{D}$; composition and first-order quantification over elements are used The symbols 0 and the names of states denote constants, with no arithmetic meaning. Finite conjunctions and disjunctions indexed by states, symbols, or transitions are abbreviations for first-order formulas.

2 Multidimensional arrays: the tiling construction

Consider the plane tiling problem. There are finitely many colours $1, \ldots, n$ and, for each colour m, sets R_m, U_m of colours allowed to its right and above. Whether $\mathbb{Z}^2$ admits such a colouring is undecidable; this is the domino problem [2].

Take distinct colour constants $x_1, \ldots, x_n$, a unary function f, and a two-dimensional array g. Require f to be a bijection:

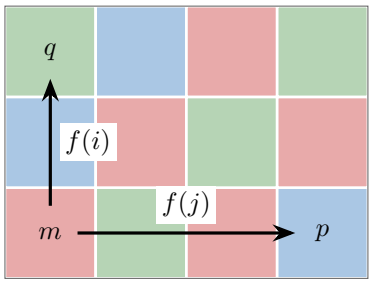
$$\forall i, j \ (f(i) = f(j) \Rightarrow i = j), \qquad \forall k \ \exists i \ (f(i) = k).$$

Require g to take a colour value and satisfy all local rules:

$$\forall i, j \quad \bigvee_{m=1}^{n} g[i, j] = x_m,$$

$$\forall i, j \quad \bigwedge_{m=1}^{n} \left(g[i, j] = x_m \Rightarrow \left[\bigvee_{p \in R_m} g[f(i), j] = x_p \right] \wedge \left[\bigvee_{p \in U_m} g[i, f(j)] = x_p \right] \right).$$

This statement imposes the restrictions we want on the assignment of colors g



An illustrative patch of the index array

Two cells above
$g[i, j] = x_m, \quad g[f(i), j] = x_q$
Required: $q \in U_m$

Three cells to the right
$g[i, f(j)] = x_p$
Required: $p \in R_m$

Figure 1: This is to show that f need not actually be the successor

A valid tiling gives a model on $\mathbb{Z}$: interpret $f(i) = i + 1$, choose distinct colour constants, and let g record the tiling. Conversely, from a model choose any $a \in \mathbb{D}$ and define the colour $h(u, v)$ by

$$g[f^u(a), f^v(a)] = x_{h(u,v)} \qquad (u, v \in \mathbb{Z}).$$

Negative powers exist because f is bijective. The local rules immediately give $h(u + 1, v) \in R_{h(u,v)}$ and $h(u, v + 1) \in U_{h(u,v)}$. Thus h is a valid tiling. Finite cycles of f cause no difficulty: they simply produce a periodic tiling.

3 Encoding stacks using functions

Let Γ be a finite stack alphabet. For each $\alpha \in \Gamma$, take an injective function $f_\alpha : \mathbb{D} \to \mathbb{D}$. Require their images to be pairwise disjoint and to avoid 0:

$$\forall x, y \; f_\alpha(x) \neq f_\beta(y) \quad (\alpha \neq \beta), \qquad \forall x \; f_\alpha(x) \neq 0.$$

For a word written from bottom to top, define

$$\mathrm{enc}(\varepsilon) = 0, \qquad \mathrm{enc}(\alpha_1 \cdots \alpha_k) = f_{\alpha_k}(\cdots f_{\alpha_2}(f_{\alpha_1}(0)) \cdots).$$

Lemma 1. *The map* enc *is a bijection between* Γ^* *and the values obtained by composing the* f_α *starting from* 0*.*

Proof. Surjectivity onto this set is immediate. No nonempty word maps to 0. If two nonempty words have the same encoding, disjointness of the images forces their final symbols to agree. Injectivity of the corresponding function then lets us cancel that symbol. Repeating this argument proves that the two words are identical. □

For each α, introduce a unary function g_α satisfying

$$\forall x \quad g_\alpha(f_\alpha(x)) = x.$$

This also implies injectivity of f_α. Pushing α applies f_α; popping α applies g_α, guarded by

$$\mathrm{Top}_\alpha(z) := \exists u \; (f_\alpha(u) = z).$$

The values of g_α outside the image of f_α do not matter..

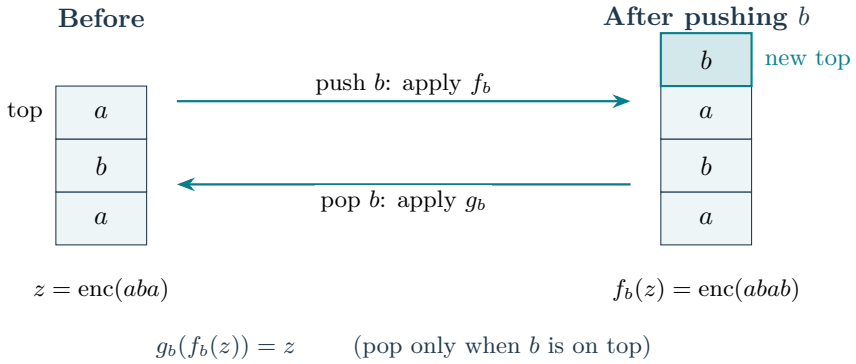


Figure 2: A stack is stored in one domain element. Its top symbol is the outermost function in its encoding.

These functions exist on every infinite domain, assuming the Axiom of Choice. Fix $0 \in \mathbb{D}$, a bijection $l : \mathbb{N} \times \mathbb{D} \to \mathbb{D}$, and a bijection $h : \mathbb{D} \to \mathbb{D} \setminus \{0\}$. If $\Gamma = \{\alpha_1, \ldots, \alpha_n\}$, set $f_{\alpha_i}(x) = h(l(i, x))$. Injectivity and disjointness follow immediately. Each function also has a left inverse g_α on its image, extend it arbitrarily everywhere else.

For $\mathbb{D} = \mathbb{N} = \{0, 1, 2, \ldots\}$, the simpler choice

$$f_{\alpha_i}(x) = nx + i, \qquad 1 \leq i \leq n,$$

suffices.

4 PDAs: Everything Everywhere All At Once

Given two PDAs A_1, A_2 over the same input alphabet, it is undecidable whether their languages have empty intersection [1]. We construct a sentence Φ_{A_1,A_2} such that

$$\Phi_{A_1,A_2} \text{ is satisfiable} \iff L(A_1) \cap L(A_2) = \varnothing.$$

We allow nondeterminism and ε-transitions, and use acceptance by final state after the entire input has been read. Each stack starts with a permanent bottom marker $\perp_j$.

4.1 Synchronising the two PDAs

A transition of A_j has the form $\delta = (p, a, \alpha, q, \beta)$. It reads $a \in \Sigma \cup \{\varepsilon\}$, changes state from p to q, and replaces top symbol α by the word β. Let $\mathcal{T}$ consist of:

1. pairs of transitions that read the same letter in Σ;
2. an ε-transition of A_1 paired with an idle step of A_2;
3. an idle step of A_1 paired with an ε-transition of A_2.

Any two finite runs on the same word can be obtained this way: perform their ε-moves separately and synchronise the letter-consuming moves.

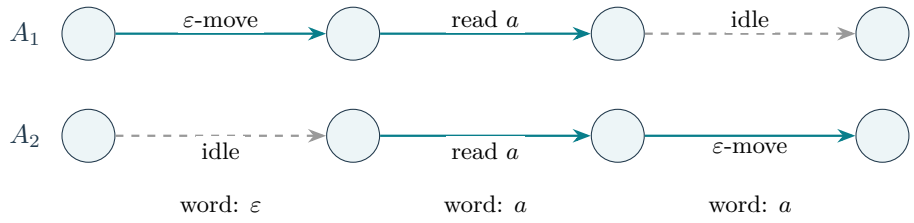


Figure 3: Three joint steps.

4.2 Spreading the good word

We pick an element r to be the root, now for each possible joint transition choice τ, introduce a unary function range disjoint injective B_τ. Applying B_τ extends a history by that choice. A sequence $h = \tau_1 \cdots \tau_k$ is represented by

$$x_h = B_{\tau_k}(\cdots B_{\tau_1}(r)\cdots), \qquad x_\varepsilon = r.$$

Each joint choice either reads the same letter in both automata or makes an ε-move in one while the other waits. Write $\lambda(\tau)$ for that letter, or ε. The input word associated with a history is

$$w(h) = \lambda(\tau_1)\cdots\lambda(\tau_k).$$

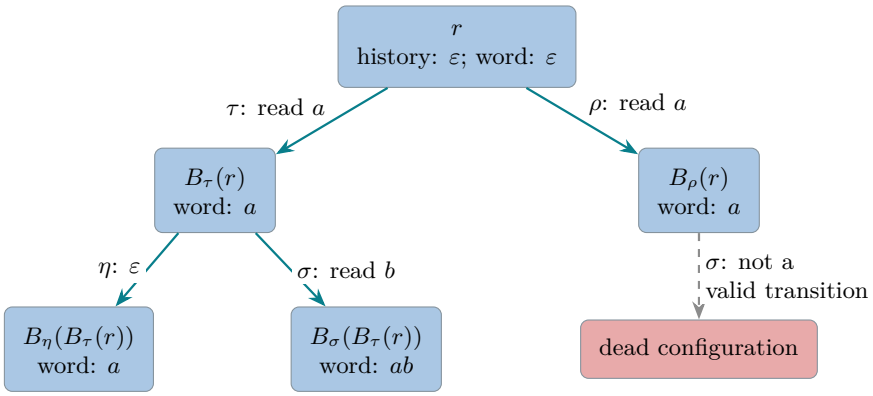


Figure 4: Blue nodes represent legal computation histories. The red node exists in the tree but cannot be reached by legal automaton moves.

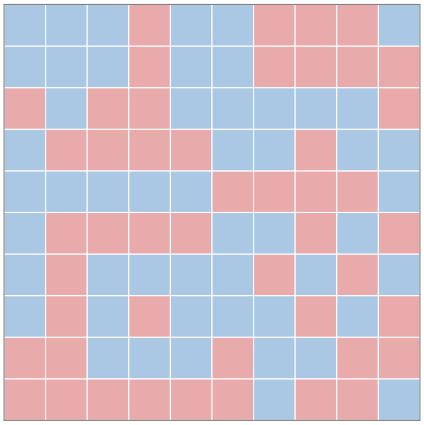


Figure 5: We dont make any assumptions on exact configuration of blue nodes.

At a node x, unary arrays Q_1, Q_2, S_1, S_2 store the two states and encoded stacks. Introduce nonaccepting dead states d_1, d_2, distinct from the ordinary state constants, and require $Q_j(x) \in \mathcal{Q}_j \cup \{d_j\}$. All state constants are pairwise distinct.

Let F_β denote the stack term obtained by pushing the symbols of β in order; $F_\varepsilon(z) = z$. For a transition $\delta = (p, a, \alpha, q, \beta)$ of A_j, define

$$G_{j,\delta}(x) := Q_j(x) = p \wedge \mathrm{Top}_\alpha(S_j(x)),$$
$$U_{j,\delta}(x, y) := Q_j(y) = q \wedge S_j(y) = F_\beta(g_\alpha(S_j(x))).$$

For an idle step, let $G_{j,\mathrm{id}}(x)$ be true and let

$$U_{j,\mathrm{id}}(x, y) := Q_j(y) = Q_j(x) \wedge S_j(y) = S_j(x).$$

4.3 The sentence and its correctness

For $\tau = (\delta_1, \delta_2)$, including idle steps, abbreviate

$$G_\tau(x) := G_{1,\delta_1}(x) \wedge G_{2,\delta_2}(x),$$
$$U_\tau(x, y) := U_{1,\delta_1}(x, y) \wedge U_{2,\delta_2}(x, y),$$
$$\mathrm{Dead}(y) := Q_1(y) = d_1 \wedge Q_2(y) = d_2 \wedge S_1(y) = 0 \wedge S_2(y) = 0.$$

For every $\tau \in \mathcal{T}$, require

$$\forall x \left[(G_\tau(x) \Rightarrow U_\tau(x, B_\tau(x))) \wedge (\neg G_\tau(x) \Rightarrow \mathrm{Dead}(B_\tau(x))) \right].$$

Initialise the root with the initial states q_j^0 and bottom markers:

$$\bigwedge_{j=1}^{2} \big(Q_j(r) = q_j^0 \wedge S_j(r) = f_{\perp_j}(0) \big).$$

Finally, exclude simultaneous acceptance everywhere:

$$\forall x \, \neg \left[\left(\bigvee_{p \in F_1} Q_1(x) = p \right) \wedge \left(\bigvee_{q \in F_2} Q_2(x) = q \right) \right].$$

The conjunction of these requirements, the stack axioms, branching axioms, and state constraints is Φ_{A_1,A_2}.

Theorem 1. *Φ_{A_1,A_2} is satisfiable if and only if $L(A_1) \cap L(A_2) = \varnothing$.*

Proof. **A model implies empty intersection.** Suppose a word were accepted by both automata. Choose their finite accepting runs and synchronise them into a history $h = \tau_1 \cdots \tau_k$. Starting at r, follow the corresponding branching functions. Initialisation and the guarded updates uniquely identify the states and stacks of the two runs at every node, by induction on the history length. The final node therefore has both states accepting, a contradiction. This also handles an empty history: the accepting node would be r itself.

Empty intersection gives a model. Use the construction from Section 2, and note by construction of the sentence simulating the PDAs correctly satisfy all the constraints simultaneously. □

5 Two-stack machines

A deterministic two-stack machine has finite control and two stacks. Blank-input nonhalting is undecidable for deterministic two-stack machines. We reduce to it here.

5.1 Counting without counting

Lemma 2. *If $c, d : \mathbb{D} \to \mathbb{D}$ are injective and have disjoint images, then*

$$c^i(d(x)) = c^j(d(y)) \quad \iff \quad i = j \text{ and } x = y.$$

Proof. If $i > j$, cancel j applications of c. This gives $c^{i-j}(d(x)) = d(y)$, contradicting disjointness of the images. The case $j > i$ is symmetric. Thus $i = j$, and injectivity of c and d gives $x = y$. The reverse implication is immediate. □

In the reduction, we write $T_+ = c$ for advancing time and $T_0 = d$ for choosing a timeline start.

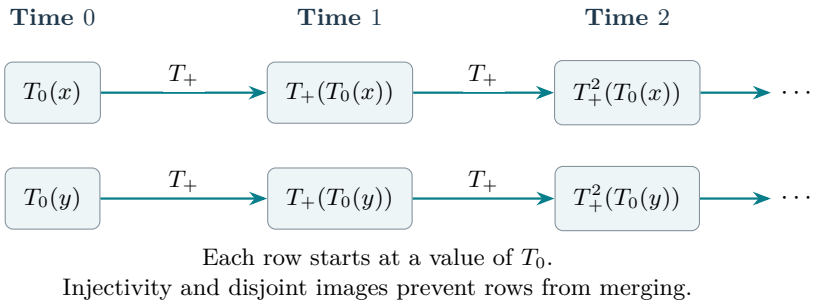


Figure 6: Independent timelines .

5.2 The first-order sentence

Fix a deterministic two-stack machine M with initial state s, halting states $F \subseteq Q$, and transition set T. Both stacks start empty.

Use unary functions T_+, T_0, the stack functions f_α, g_α, and the arrays q, S_1, S_2. State names are pairwise distinct constants. Let Φ_M be the conjunction of the following clauses.

(1) Time functions. Both T_+, T_0 are injective and their images are disjoint:

$$\bigwedge_{T \in \{T_0, T_+\}} \forall x, y\ (T(x) = T(y) \Rightarrow x = y), \qquad \forall x, y\ T_+(x) \neq T_0(y).$$

(2) Stack functions. For every stack symbol α and every distinct pair α, β, impose

$$\forall x\ g_\alpha(f_\alpha(x)) = x, \qquad \forall x\ f_\alpha(x) \neq 0, \qquad \forall x, y\ f_\alpha(x) \neq f_\beta(y).$$

(3) States. Impose distinctness of the state constants and

$$\forall i \quad \bigvee_{p \in Q} q[i] = p.$$

(4) Initial configurations. Initialise the state and both stacks at the beginning of every timeline:

$$\forall x \quad q[T_0(x)] = s \ \wedge\ S_1[T_0(x)] = 0 \ \wedge\ S_2[T_0(x)] = 0.$$

(5) Transitions. Let $\square$ denote an empty-stack test, not a stack symbol, and put

$$\mathrm{Top}_\square(z) := (z = 0), \qquad \mathrm{Top}_\alpha(z) := \exists u\ (f_\alpha(u) = z).$$

For $\tau = (p, a_1, a_2, r, \mathrm{op}_1, \mathrm{op}_2) \in T$, define

$$\begin{aligned} G_\tau(i) &:= q[i] = p \wedge \mathrm{Top}_{a_1}(S_1[i]) \wedge \mathrm{Top}_{a_2}(S_2[i]), \\ U_\tau(i) &:= q[T_+(i)] = r \\ &\qquad \wedge S_1[T_+(i)] = \mathrm{op}_1(S_1[i]) \wedge S_2[T_+(i)] = \mathrm{op}_2(S_2[i]). \end{aligned}$$

Here each operation abbreviates f_α, g_α, or the identity, with pops permitted only under the corresponding top-symbol guard. Require

$$\forall i \quad \bigvee_{\tau \in T} \big(G_\tau(i) \wedge U_\tau(i)\big).$$

Thus some transition must actually be taken.

(6) Nonhalting. Finally, require

$$\forall i \quad \bigwedge_{p \in F} q[i] \neq p.$$

6 Correctness of the reduction

Theorem 2. *M does not halt from its initial configuration with two empty stacks if and only if Φ_M is satisfiable.*

Proof. **Suppose Φ_M is satisfiable.** Choose any $x \in \mathbb{D}$ and consider

$$T_0(x),\ T_+(T_0(x)),\ T_+^2(T_0(x)), \ldots.$$

By Lemma 2, these indices are distinct. Clause (4) puts the initial configuration at the first index. Inductively, if the element at an index encode the actual machine configuration, the next element along the timeline describe the actual computation of M.

Clause (6) excludes every halting state along the chain. Hence M does not halt.

Suppose M does not halt. Let its infinite computation be

$$(q_0, w_{1,0}, w_{2,0}),\ (q_1, w_{1,1}, w_{2,1}), \ldots, \qquad (q_0, w_{1,0}, w_{2,0}) = (s, \varepsilon, \varepsilon).$$

Take an infinite domain $\mathbb{D}$ and construct $T_+ = c$, $T_0 = d$ as in Section 5.1 so that every element has a unique representation $T_+^j(T_0(y))$. Independently choose the stack functions as in Section 2 and interpret the finitely many state constants as distinct elements of $\mathbb{D}$.

For every $y \in \mathbb{D}$ and $j \in \mathbb{N}$, define

$$\begin{aligned} q[T_+^j(T_0(y))] &= q_j, \\ S_1[T_+^j(T_0(y))] &= \mathrm{enc}(w_{1,j}), \\ S_2[T_+^j(T_0(y))] &= \mathrm{enc}(w_{2,j}). \end{aligned}$$

Uniqueness of the representation makes these total arrays well defined. Each timeline carries the same computation. The time and stack clauses hold by construction, initialisation holds at $j = 0$, and every adjacent pair of configurations satisfies a transition clause. Since the computation never halts, clause (6) also holds. Thus this is a model of Φ_M. □

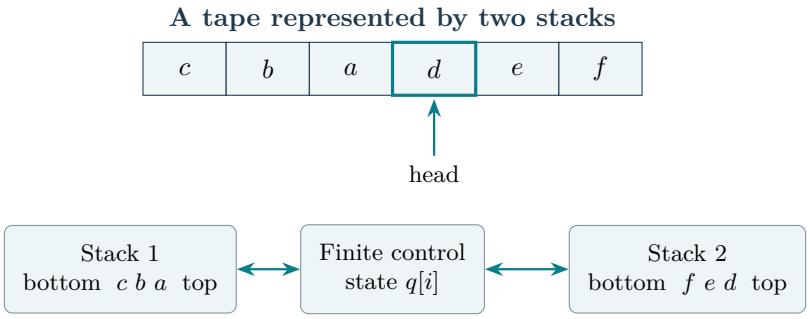


Figure 7: we simulate a right move by a push and a pop, just a push suffices for extreme moves.

References

[1] Arto Salomaa. *On a Homomorphic Characterization of Recursively Enumerable Languages*. Annales Academiae Scientiarum Fennicae, Series A I, Mathematica, 525, 1973. doi:10.5186/aasfm.1973.525.

[2] Robert Berger. *The Undecidability of the Domino Problem*. Memoirs of the American Mathematical Society, 66, 1966. doi:10.1090/memo/0066.

[3] Börger, Egon and Grädel, Erich and Gurevich, Yuri, *The Classical Decision Problem*, Perspectives in Mathematical Logic, doi:10.1007/978-3-642-59207-2